TECHNICAL REPORT | 2026

Quant Learning OS (QLOS)

Deep Learning Week 2026 Submission

DOCUMENT TYPE	DATE	CLASSIFICATION	VERSION
Technical Architecture Report	March 3, 2026	Submission — Public	1.0

Every technical claim in this document maps to a specific file path, endpoint, or log entry in the accompanying repository.
Responsible AI gaps are documented explicitly.

Quant Learning OS (QLOS)

> ***Deep Learning Week 2026 Submission*** *Theme: Responsible AI in Education*

The Problem We're Solving

Four questions that every student on a digital learning platform cannot answer:

1. **"What am I genuinely weak at versus just making careless mistakes?"** — Platforms store right/wrong. They don't store *why*.
2. **"Am I improving, stagnating, or regressing over time?"** — Score history shows numbers. It doesn't surface trends.
3. **"What should I focus on if I only have two hours tonight?"** — Recommendations are generic. They don't account for time constraints or compound weaknesses.
4. **"Why do I keep failing the same type of question?"** — There is no failure-pattern loop. Every session starts fresh.

QLOS is built to answer all four — directly, with evidence you can trace — for students targeting quantitative finance roles. It models learning state continuously across quiz performance, interview practice, live trading behavior, and resume evidence, then uses that composite to generate ranked recommendations, adaptive study plans, and grounded coaching responses.

Learning is non-linear. QLOS accounts for gaps (fallback signals when data is sparse), bursts (spaced repetition and mastery tracking), and drift (behavioral trends over rolling 10-day windows). Every AI action is auditable by design.

Table of Contents

1. Judging Criteria
2. What Makes QLOS Different
3. How It Maps to the Problem Statement
4. Architecture
5. Core Innovations
6. Responsible AI — First-Class, Not an Afterthought
7. Full Feature Inventory
8. Technical Implementation
9. Test Cases and Verification
10. Results and Findings
11. Observations and Honest Limitations
12. API Reference

13. Local Development

14. Repository Map

---

0. Judging Criteria

INNOVATION & CREATIVITY

QLOS approaches learning analytics from a direction no general platform takes: treating trading behavior, interview performance, resume evidence, and quiz history as a single unified learning signal rather than isolated features.

Novel ideas: - **Behavioral execution analytics** derived from live paper-trading orders — fat-finger risk, revenge-trade detection, stop-loss discipline, slippage sensitivity — fed directly into the academic readiness composite. No other learning platform connects live market behavior to study recommendations. (→ Innovation 2) - **Mistake taxonomy persistence** — every wrong answer is annotated as `Conceptual`, `Careless`, or `Implementation` before the answer is revealed. This distinction drives fundamentally different study plan types. (→ Innovation 4) - **Firm-specific interview scoring** — the same answer receives a different weighted score under Jane Street's reasoning-first rubric vs. Citadel's execution-first rubric, reflecting real hiring calibration differences. (→ Innovation 7) - **DistilBERT query router** — a fine-tuned classifier routes coaching queries to the appropriate model tier by complexity, trained on auto-labeled seed data with zero manual labeling cost. (→ Innovation 6)

TECHNICAL IMPLEMENTATION

The stack is deliberate: Next.js API routes orchestrate Python subprocesses on-demand — no persistent Python server, no port management, one `npm run dev` command starts everything.

Technical choices and why: - **Python via `child_process.spawn`** — keeps the deployment surface minimal while allowing full access to PyTorch, FAISS, scikit-learn, and OpenAI function-calling within the same repo. (→ §7 Technical Implementation) - **OpenAI function-calling agent with deterministic fallback** — the study planner runs a structured 5-step tool workflow (`rank_readiness_gaps` → `get_drill_templates` → `build_seed_plan` → `estimate_weeks_to_target` → `score_plan`). The LLM writes every task description itself; `hoursPerWeek` and `targetRole` are read directly from the user's profile. If the API call fails or returns malformed JSON, a pure-Python fallback executes the same gap-ranking logic. Students always receive a usable plan. (→ Innovation 3) - **FAISS RAG with LLM-as-a-Judge** — coaching answers are grounded in uploaded documents; each factual claim is evaluated as `supported`/`partial`/`unsupported`/`uncited` before display. (→ Innovation 5) - **Append-only NDJSON audit log** — every study plan generation is recorded unconditionally, including failures. The log is never overwritten, enabling offline audit and bias review. (→ §5 Responsible AI) - **SQLite for all learning state** — quiz progress (including partial in-session state), interview history, resume analyses, and conversation history all persist across sessions with no additional infrastructure.

Code quality signals: output sanitization and schema validation before every API response; confidence-threshold fallback in the query router; rolling trend computation separated from score computation; no shared mutable state between Python subprocesses.

IMPACT & VIABILITY

The problem is real: students preparing for quant roles have no platform that closes the loop between what they study, how they trade, and how they interview. The four problem questions in the introduction are not hypothetical — they are the questions every QLOS user can now answer directly.

Scalability path: - Swap SQLite for Postgres — schema is already normalized, no structural changes needed - Replace `child_process.spawn` with a task queue (Celery, BullMQ) for concurrent users — the subprocess boundary is already clean - The audit log format (NDJSON with `auditId`) is production-ready for ingestion into

any log aggregator (Datadog, Splunk, BigQuery) - The readiness composite formula is parameterizable — weights can be adjusted per institution or per role without code changes

Meaningful problem addressed: the gap between academic preparation and job-readiness for quantitative roles is well-documented. QLOS gives students a single instrument that tracks both dimensions simultaneously and generates targeted, time-bounded study plans grounded in their actual behavioral data — not generic advice.

---

1. What Makes QLOS Different

Most platforms: one score, one recommendation, no transparency.

PROBLEM QUESTION	BASELINE PLATFORM	QLOS
Conceptual weakness vs. careless mistake?	Not tracked	Per-question mistake taxonomy: `Conceptual` / `Careless` / `Implementation` + confidence level, persisted and surfaced in next session
Am I improving or regressing?	Score number only	Rolling 10-day trend per behavioral signal; readiness momentum computed from latest interview + trading history
What to focus on with limited time?	"Review Chapter 3"	Agentic planner allocates specific minutes to weakest components given `hoursPerWeek` budget; sessions have measurable targets
Why do I keep failing the same question type?	No loop	Mistake type breakdown feeds study-plan generation; pattern surfaces recommended session type (concept review vs. drill vs. debugging practice)
Is the AI recommendation trustworthy?	Opaque text output	Every plan has an `auditId`, a `toolTrace`, and is linked to the signal that triggered it; coaching answers carry citation IDs and claim-level support scores
What if the AI fails?	System error	Deterministic fallback: every LLM-driven endpoint returns a usable result even with no API key

---

2. How It Maps to the Problem Statement

"HOW LEARNING INTERACTION DATA CAN BE STRUCTURED, INTERPRETED, AND TRACKED OVER TIME"

QLOS captures and structures five distinct interaction streams:

STREAM	WHAT IS CAPTURED	WHERE STORED	HOW IT FEEDS STATE
Quiz attempts	answer, confidence (`Low/Med/High`), mistake type, timestamp, in-progress partial state	SQLite `quiz_progress`	Theory component of readiness composite
Interview sessions	per-answer score, firm-weighted score, question category, session history	SQLite `interview_results`	Implementation + Communication components
Trading behavior	order type, qty, symbol, timing, fill price — from live Alpaca paper account	Alpaca API → computed on-demand	Execution component (4 behavioral signals)
Resume evidence	extracted skills, evidence strength, career highlights, readiness score	SQLite `resume_analyses`	Theory component baseline
Coaching sessions	conversation history, document queries, citation requests	SQLite `conversations`	Informs coaching context

Tracking over time is explicit: behavioral signals compute trends comparing the last 5 trading days against the prior 5. Interview histories are stored as ordered rows. Quiz progress accumulates per-user per-quiz.

"HOW AI-GENERATED INSIGHTS ARE DELIVERED IN A CLEAR, EXPLAINABLE, AND ACTIONABLE WAY"

Three concrete mechanisms:

1. Composite with visible components. The readiness score is not a black box. It is broken down into Theory (30%), Implementation (25%), Execution (30%), Communication (15%) with the source signal named for each. Users can see which component is weakest and why.

2. Evidence-linked recommendations. Every recommendation on the dashboard carries a `source` tag: `/profile/resume`, `/profile/interview`, or `/trade`. Users know *why* the system is recommending something.

3. Tool traces on study plans. When the agentic planner generates a study plan, the response includes a `toolTrace` array showing every tool the agent called, what arguments it used, and what it returned. The `auditId` links this to the append-only audit log. Users and reviewers can verify the plan's provenance.

"HOW THE SYSTEM ADAPTS TO LONG-TERM CHANGES IN BEHAVIOR, INCLUDING INACTIVITY OR ACCELERATED PROGRESS"

- **Inactivity:** When behavioral data is absent (no Alpaca orders), the Execution component defaults to a neutral 50/100 and the explanation notes "No order history yet." The composite still functions — it doesn't break when data is sparse.
- **Behavioral drift:** Trend indicators (`up`/`flat`/`down`) on each behavioral signal compare the most recent 5 trading days to the prior 5. A student who was placing stop orders but stopped will see `stopLossTrend: "down"` before the overall score drops.
- **Accelerated progress:** If interview scores jump, the Implementation component rises, the composite rises, and the study plan allocates less time to that component in the next generation.
- **Recommendation shift:** Recommendations are generated dynamically from the current state — they are not cached. A student who fixes their weakest component gets different recommendations next time.

3. Architecture

```
Browser (Next.js App Router, React 19, TypeScript)
    |
    └─▶ 23 Next.js API routes
            |
            ├─▶ SQLite (local)
            |       users, quiz_progress, conversations,
            |       interview_results, resume_analyses
            |
            ├─▶ Local filesystem
            |       data/resumes/          uploaded resume files
            |       data/responsible-ai/   append-only audit logs (NDJSON)
            |
            ├─▶ Python subprocesses  [spawned on-demand via child_process.spawn]
            |       study_plan_agent.py        agentic readiness planner
            |       interview_llm.py           firm-specific LLM interviewer
            |       resume-analysis/api.py     skill extraction (70+ taxonomy)
            |       rag_cli.py                 FAISS RAG + claim evaluation
            |       smart_beta_service.py      factor-regression analysis
            |       router/router_cli.py       DistilBERT query router
            |
            └─▶ External APIs
                    OpenAI (gpt-4o, gpt-4o-mini)
                    Alpaca Paper Trading
                    yfinance / FRED / Ken French
```

One process, no server sprawl. All Python workloads spawn on-demand from Next.js API routes. A single `npm run dev` starts the entire system. No port management, no separate Python servers, no coordination overhead.

---

4. Core Innovations

INNOVATION 1: MULTI-SIGNAL LEARNING STATE MODEL

What it solves: Problem question 1 (weakness type) and 3 (what to prioritize).

The readiness composite aggregates four evidence streams with explicit weights:

```
Readiness = 0.30 × Theory + 0.25 × Implementation + 0.30 × Execution + 0.15 × Communication
```

COMPONENT	SOURCE SIGNAL	FALLBACK WHEN DATA ABSENT
Theory (30%)	Resume analysis `overall_score × 10`	50 (neutral)
Implementation (25%)	Interview `avg_score × 20`	50
Execution (30%)	Behavioral composite from Alpaca orders	50
Communication (15%)	Interview strong-answer ratio blend	50

This is more than a weighted average — each component exposes its source. A learner who scores 85% on quizzes but never places stop orders receives Theory ≈ 85, Execution ≈ 15, and a composite around 51. The system correctly identifies the true weak point. The quiz-only signal would have returned 85.

Files: `components/profile/ReadinessTab.tsx`, `app/page.tsx`

INNOVATION 2: BEHAVIORAL EXECUTION ANALYTICS

What it solves: Problem question 4 (why do I keep failing the same pattern?) — extended to trading execution discipline.

`app/api/alpaca/behavioral/route.ts` derives four behavioral signals from raw order history. Each signal has a trend direction computed by comparing the last 5 trading days to the prior 5:

Fat-Finger Risk (0–100, lower is better) Identifies orders where `qty > 2× the user's own median order qty`. Formula: `(flagged_orders / total_orders) × 200`, capped at 100. A student consistently oversizing positions sees this score rise and receives a coaching note.

Revenge-Trade Risk (0–100, lower is better) Detects market orders placed within 5 minutes of a prior fill on the same symbol — the classic panic re-entry pattern. Formula: `(burst_count / filled_orders) × 500`, capped at 100.

Stop-Loss Discipline (0–100, higher is better) Among all active trading days, what percentage also had a stop or stop_limit order fill? Students who trade without protection score 0 immediately.

Slippage Sensitivity (0–100, higher is better) Average basis-point gap between `filled_avg_price` and `limit_price` on limit orders. Inverted: better execution discipline → higher score.

These feed directly into the Execution component of the readiness model. Behavioral pattern → readiness impact → study plan adjustment is a closed loop.

File: `app/api/alpaca/behavioral/route.ts`

INNOVATION 3: AGENTIC STUDY PLANNING WITH FULL AUDIT TRAIL

What it solves: Problem question 3 (what to focus on with limited time) + the Responsible AI requirement for transparent AI actions.

`ml-development/readiness-agent/study_plan_agent.py` runs an OpenAI function-calling loop with **five tools** in a structured 5-step workflow. The LLM is instructed to call them in order and to write all task descriptions itself — it never copies recommendation titles.

STEP	TOOL	PURPOSE
1	`rank_readiness_gaps`	Sort components by severity (lowest score, widest gap)
2	`get_drill_templates(component_key, difficulty)`	Fetch 2–3 typed drill templates per weak component as structural inspiration
3	`build_seed_plan(top_n)`	Return budget context: `suggestedMinutes`, `difficultyLevel`, `evidenceContext` per component — the LLM then writes its own task text
4	`estimate_weeks_to_target(component_key, current_score, target_score)`	Project weeks-to-target using `WEEKLY_GAIN_RATE` constants scaled by the learner's `hoursPerWeek` (capped at 1.5×)
5	`score_plan(sessions)`	Validate the draft: budget compliance (≤110%), coverage ratio, session count 1–6; returns score 0–100. Agent revises once if score < 70

Personalisation: `hoursPerWeek` and `targetRole` are read directly from the authenticated user's profile (not hardcoded). The weekly minute budget and difficulty tier (`beginner` / `intermediate` / `advanced`) are surfaced at the top of the user prompt so the model can't miss them.

LLM-authored tasks: `build_seed_plan` no longer returns pre-written session strings. It returns context (`evidenceContext`, `suggestedMinutes`, `instruction`). Every task description in the final plan is written by the LLM from scratch. The `source: "agent"` label in the response confirms this path was taken.

`weeklyOutlook`: The response optionally includes a multi-week projection array (up to 8 weeks) with `{ week, focus, milestone, estimatedMinutes }`, giving students a forward-looking schedule in addition to the current week's sessions.

Deterministic fallback path: If the API key is absent, a tool fails, or the model returns malformed JSON, the fallback planner executes the same gap-ranking logic in pure Python. The user receives an actionable plan regardless. The `source` field indicates `"agent"` vs `"fallback"` — nothing is hidden.

Audit record (written on every call — success, fallback, and error):

```
{
  "auditId": "uuid",
  "startedAt": "2026-03-03T10:22:11.000Z",
  "status": "ok",
  "model": "gpt-4o-mini",
  "requestSummary": {
    "composite": 58,
    "hoursPerWeek": 10,
    "targetRole": "Quant Research",
    "breakdown": [{"key": "theory", "score": 45}, ...],
    "recommendationCount": 3
  },
  "responseSummary": {
    "source": "agent",
    "sessionCount": 4,
    "weeklyMinutes": 600,
    "toolCount": 5
  },
  "toolTrace": [
    {"step": 1, "toolName": "rank_readiness_gaps",       "outputSummary": "Top gap: Theory
Mastery (55 points)", "status": "ok"},
    {"step": 2, "toolName": "get_drill_templates",        "outputSummary": "2 templates for
theory at intermediate level", "status": "ok"},
    {"step": 3, "toolName": "build_seed_plan",            "outputSummary": "Context for 3
components, 600m budget, difficulty=intermediate", "status": "ok"},
    {"step": 4, "toolName": "estimate_weeks_to_target",   "outputSummary": "Estimated 9.9 weeks
to close 37-point gap in theory", "status": "ok"},
    {"step": 5, "toolName": "score_plan",                 "outputSummary": "Plan score: 95/100.
BudgetOk=true, Coverage=0.75", "status": "ok"}
  ]
}
```

Every plan is inspectable. Every failure is logged. No AI action on a student's learning trajectory goes unrecorded.

Files: `ml-development/readiness-agent/study_plan_agent.py`, `app/api/profile/readiness/study-plan/route.ts`, `data/responsible-ai/study-plan-tool-audit.ndjson`

INNOVATION 4: MISTAKE TAXONOMY AND RECOVERY PERSISTENCE

What it solves: Problem questions 1 and 4 — distinguishing error type and surfacing recurring patterns.

For every quiz question, the learner annotates: - **Confidence level:** `Low` / `Med` / `High` (before seeing the answer) - **Mistake type** if wrong: `Conceptual` / `Careless` / `Implementation`

All state — including partial in-progress answers — is persisted by `userId + quizId` in SQLite via `POST /api/learn/quiz-progress`. On re-entry, the quiz resumes at the last unanswered question. No session loss on refresh or tab close.

Why it matters for the problem statement: A student with 80% `Conceptual` errors in probability needs concept re-teaching. A student with 80% `Careless` errors needs timed drill practice. A student with 80% `Implementation` errors needs debugging exercises. These are different study plans. Without the taxonomy, the system cannot make the distinction.

Files: `app/learn/quiz/[id]/page.tsx`, `lib/quiz-progress.ts`, `lib/types.ts`

INNOVATION 5: CITATION-GROUNDED COACHING WITH CLAIM EVALUATION

What it solves: The Responsible AI requirement — coaching advice must be verifiable, not just plausible.

`ml-development/rag-pipeline/rag_cli.py` implements a full RAG pipeline over uploaded documents (PDF, DOCX, TXT):

Ingestion: Chunks documents with `RecursiveCharacterTextSplitter` (500 chars, 100 overlap). Enriches each chunk with a citation ID (`c{page}.0.{paragraph}`), page number, paragraph index, text preview, and estimated bounding box. Embeds with HuggingFace `all-MiniLM-L6-v2`. Stores in FAISS.

Query: Top-5 chunk retrieval → OpenAI generation with citation-required prompt → inline references in answer (e.g., `[c1.0.3]`).

LLM-as-a-Judge: Extracts individual factual claims from the answer and scores each as `supported` / `partial` / `unsupported` / `uncited`. Aggregates into overall confidence score. This runs before the answer is displayed to the user.

A student asking about a concept from their course notes gets an answer they can verify sentence by sentence — not an answer they have to trust blindly.

Files: `ml-development/rag-pipeline/rag_cli.py`, `app/api/coaching/rag/route.ts`

INNOVATION 6: DISTILBERT QUERY COMPLEXITY ROUTER

What it solves: Cost efficiency — using the most capable model only when the query warrants it, without sacrificing quality on hard questions.

Training pipeline (all in `router/`): 1. `bootstrap_labels.py` — auto-labels ~150 seed queries (simple/medium/complex) using local Ollama, zero labeling cost 2. `prepare_data.py` — tokenizes with `distilbert-base-uncased`, creates train/val split 3. `train.py` — fine-tunes for 3-class text classification (≈5 min on CPU)

Inference (`router/router.py`): - Confidence threshold: 0.70 — below this, defaults to `"medium"` (safe fallback, never mis-routes a hard question to a weak model) - Route map: `simple` → `gpt-4o-mini`, `medium` → `gpt-4o-mini`, `complex` → `gpt-4o` - Router label and confidence returned in response headers (`X-Router-Label`, `X-Router-Confidence`)

Example classifications:

QUERY	EXPECTED LABEL	RATIONALE
"What is a Sharpe ratio?"	simple	Definitional retrieval
"Explain Kalman filters for pairs trading"	medium	Concept + application
"Derive the Black-Scholes PDE from Itô's lemma"	complex	Multi-step proof

Files: `router/router.py`, `router/train.py`, `app/api/chat/route.ts`

INNOVATION 7: FIRM-SPECIFIC INTERVIEW SCORING

What it solves: Generic interview prep produces generic performance. Real quant firms have distinct interview styles and distinct hiring priorities.

`ml-development/interview-pipeline/interview_llm.py` parameterizes the entire interviewer by firm with a persona guide and a 4-dimension hiring-manager priority profile:

FIRM	PRIORITY 1 (WEIGHT)	PRIORITY 2 (WEIGHT)	PRIORITY 3 (WEIGHT)	PRIORITY 4 (WEIGHT)
Jane Street	Quality of reasoning (0.35)	Collaborative problem solving (0.25)	Coachability (0.20)	Communication clarity (0.20)
Citadel	Delivery & ownership (0.30)	Performance-minded engineering (0.25)	Execution under pressure (0.25)	Decision quality (0.20)
HRT	Systems/performance depth (0.35)	Debugging (0.25)	Low-level judgment (0.25)	Independent execution (0.15)
Two Sigma	Statistical rigor (0.30)	Production discipline (0.25)	Trade-off judgment (0.25)	Cross-functional execution (0.20)
D.E. Shaw	Structured fundamentals (0.30)	Long-horizon code quality (0.25)	Communication precision (0.25)	Technical-business judgment (0.20)
SIG	Decision quality under uncertainty (0.30)	Teamwork (0.25)	Reliable implementation (0.25)	Commercial awareness (0.20)

Weighted score = Σ(priority_score × weight) per firm per answer. The same answer receives different scores under different firm profiles — reflecting real hiring calibration differences.

File: `ml-development/interview-pipeline/interview_llm.py`

---

5. Responsible AI — First-Class, Not an Afterthought

Responsible AI is the central theme of this submission. It is implemented in code, not described in a principles document.

WHAT "RESPONSIBLE AI" MEANS IN QLOS

In the context of a student learning platform, Responsible AI has three concrete meanings:

1. **Transparency:** Students and reviewers can inspect why every recommendation was made.
2. **Reliability:** AI failures produce graceful fallbacks, never silent errors or broken experiences.
3. **Verifiability:** Every AI-generated output — study plan, coaching answer, interview score — can be traced to a source and checked.

IMPLEMENTED CONTROLS (WITH CODE LOCATIONS)

Append-only audit log for all study-plan generation File: `data/responsible-ai/study-plan-tool-audit.ndjson` Every call to `/api/profile/readiness/study-plan` appends a record unconditionally — success, fallback, and error. Records include: `auditId`, `startedAt`, `model`, request summary, response summary, full `toolTrace[]`, and errors. The file is never overwritten. This enables offline review, bias detection, and debugging. Code: `app/api/profile/readiness/study-plan/route.ts:68-71`

Structured tool trace per agentic planning run Every tool call in the planning agent captures: `step`, `toolName`, `arguments`, `outputSummary`, `status`, `invokedAt`. Returned to the caller in the API response so the UI can display it. Code: `ml-development/readiness-agent/study_plan_agent.py`

Deterministic fallback on every LLM-driven endpoint Study plan, interview, resume analysis, RAG query — all have non-LLM fallback paths. Fallback source is clearly labeled in the response. Students always receive output. Code: `ml-development/readiness-agent/study_plan_agent.py` (fallback planner at bottom of file)

Evidence-linked recommendations Every recommendation on the dashboard carries a `source` field pointing to the signal that triggered it: `/profile/resume`, `/profile/interview`, or `/trade`. Students know where advice comes from.

Output sanitization and clamping Study-plan outputs are validated before response: session counts bounded, durations clamped 15–90 min, required fields checked. Malformed agent output triggers the fallback rather than surfacing garbage to the student.

Citation-required coaching RAG answers are generated with a prompt that requires inline citations. LLM-as-a-Judge then scores each factual claim before display. Students can see `supported`, `partial`, `unsupported`, or `uncited` per claim.

Prompt inspection for diagnostics `includePrompt=true` query parameter on the readiness diagnostics endpoint returns the exact system and user prompts for audit purposes.

RESPONSIBLE AI GAPS (HONEST — REQUIRED FOR PRODUCTION)

These are documented, not hidden:

- No PII redaction on conversation payloads stored in SQLite
- No explicit consent flow before cross-signal personalization activates
- No content moderation gate before LLM output is displayed to the student
- No formal bias or fairness monitoring for recommendations
- No data retention policy or automated deletion workflow
- No human-in-the-loop approval gate for high-impact study plan advice
- Auth is development-grade: localStorage session + plaintext passwords in local SQLite

These gaps are in `docs/responsible-ai-framework.md`. They represent the mandatory hardening work before any production deployment — not optional improvements.

RESPONSIBLE AI ARCHITECTURE SUMMARY

```
Student action
    |
    ▼
AI component runs
    |
    ├─▶ Output sanitized + schema validated
    |
    ├─▶ Audit record written (unconditional)
    |        auditId, toolTrace, source, model, request/response summary
    |
    ├─▶ Source tagged on every recommendation
    |        /profile/resume | /profile/interview | /trade
    |
    ├─▶ Citations required for coaching answers
    |        per-claim support evaluation before display
    |
    └─▶ Deterministic fallback if LLM fails
             source: "fallback" labeled in response
```

---

6. Full Feature Inventory

DASHBOARD

- Readiness composite (0–100) with four visible component scores
- Momentum trend: improving / flat / declining, derived from recent signal history
- Ranked recommended actions, each linked to the weakest component
- Activity feed with timestamped events across all modules

LEARN

- 100+ questions across 6 categories: Probability, Statistics/ML, Microstructure, Derivatives, Python, Mental Math
- Per-question confidence annotation (`Low` / `Med` / `High`) before answer reveal
- Mistake taxonomy (`Conceptual` / `Careless` / `Implementation`) for wrong answers
- In-progress quiz state persisted and recovered on re-entry
- Mastery analytics with trend sparklines
- Spaced repetition deck

INTERVIEW PRACTICE

- 6 question categories × 6 firm personas × 3/5/10 question counts
- Turn-by-turn LLM interviewer with firm-adapted style
- Per-answer score (1–5), model answer, and firm-weighted HM score
- Session summary with average score and category breakdown
- Historical results stored per user in SQLite

TRADE SIMULATOR

- Alpaca paper trading: live account balance, buying power, P&L

- Market and limit order entry (buy/sell)
- Open positions table with unrealized P&L
- Order history
- Candlestick price charts
- Four behavioral signals computed from order history
- Session-level tracking

SMART BETA ANALYSIS

- 13 ETF universe (USMV, MTUM, QUAL, VBR, IWF, HDV, VIG, DGRO, NOBL, VLUE, DGRW, SPHB, EFAV)
- 9-factor Goldman Sachs QIS framework: MKT, SMB, HML, MOM, BAB, QMJ, RMW, CARRY, ILLIQ
- OLS regression with Newey-West HAC standard errors
- Rolling 36-month beta charts
- Factor correlation heatmap (9×9)
- 24h Parquet cache for fast re-queries

COACHING ASSISTANT

- Streaming chat with conversation history
- `auto` model routing via DistilBERT router (gpt-4o-mini or gpt-4o)
- Document upload → FAISS RAG mode with citations
- Claim-level support evaluation before display

PROFILE WORKSPACE (7 TABS)

- Resume upload → skill extraction → 70+ quant skill taxonomy
- Skills Matrix with self-rating + evidence badge (Verified / Needs Evidence / At Risk)
- Readiness diagnostics with study-plan generation
- Interview history and session review
- Aspirations and career intent tracking
- Settings

NOTIFICATIONS

- Global notification center with filter pills (All, Unread, Trade, Learning, System)
- Bell icon with unread badge in header
- Deep-linking to relevant pages from notifications

---

7. Technical Implementation

PYTHON SUBPROCESS ARCHITECTURE

No persistent Python server. All Python AI workloads run on-demand via `child_process.spawn("python3", [...])` from Next.js API routes. Each subprocess: - Reads its input from stdin (JSON payload) - Writes output to stdout (JSON response) - Times out safely (45s for planning agent, 2s for query router) - Falls back gracefully on non-zero exit or timeout

This means the full application runs with `npm run dev`. No uvicorn, no port management, no service coordination.

LEARNING STATE PERSISTENCE

DATA TYPE	STORE	KEY
User accounts	SQLite `users`	userId
Quiz progress	SQLite `quiz_progress`	userId + quizId
Interview results	SQLite `interview_results`	userId + sessionId
Resume analyses	SQLite `resume_analyses`	userId
Conversations	SQLite `conversations`	userId + conversationId
UI state	localStorage	`qlos_user`, `qlos_notifications_v1`
Planning audit	NDJSON append-only file	`data/responsible-ai/study-plan-tool-audit.ndjson`

FACTOR ANALYSIS PIPELINE

`ml-development/etf-analysis/smart_beta_service.py` wraps the original `smart_beta_checks_comprehensive.py` script with: - Parquet caching (24h TTL, configurable via env var) - JSON serialization handling (NaN, Inf, None) - Dynamic ETF support via yfinance - Returns: price history (normalized to base 100), factor loadings (alpha, betas, t-stats, p-values, R^2), rolling 36-month betas, factor correlation matrix

---

8. Test Cases and Verification

All tests assume the app is running at `http://localhost:3000`. Log in with demo credentials: `alexkhoo@gmail.com` / `demo1234`.

TC-01 — BEHAVIORAL ANALYTICS: EMPTY STATE FALLBACK

Tests: Graceful degradation when Alpaca is unconfigured (non-linear learning: inactivity case).

1. Navigate to the **Dashboard** (`/`)
2. Find the **Trading Execution** section of the readiness breakdown
3. Look for all four behavioral signals (Fat-Finger Risk, Revenge Trade Risk, Stop-Loss Discipline, Slippage Sensitivity)

What to look for: All four signals show `0` with `flat` trend indicators. The explanation reads `"No order history yet."` The overall composite still renders — no error, no blank panel. The app degrades gracefully when trading data is absent.

Pass criteria: Dashboard loads without errors. Execution component displays a neutral state rather than breaking.

TC-02 — BEHAVIORAL ANALYTICS: FAT-FINGER DETECTION LOGIC

Tests: Problem question 4 — identifying a specific, repeating bad pattern.

1. Navigate to the **Dashboard** (`/`) with an Alpaca account that has order history including an outsized order (e.g., one order 5–10× the typical quantity)
2. Open the **Trading Execution** breakdown panel

What to look for: The Fat-Finger Risk score is non-zero and proportional to the fraction of oversized orders. The explanation text explicitly names the pattern: `"oversized order events detected."` The score uses the formula: `(flagged orders / total orders) × 200`, capped at 100.

Pass criteria: Risk score is mathematically correct relative to order history. Explanation text names the specific pattern rather than giving a generic message.

TC-03 — BEHAVIORAL ANALYTICS: STOP-LOSS DISCIPLINE

Tests: Discipline tracking across active trading days.

1. Navigate to the **Dashboard** (`/`) with an Alpaca account that has at least 5 active trading days
2. Open the **Trading Execution** breakdown panel

What to look for: Stop-Loss Discipline score reflects the ratio of days with stop orders placed to total active trading days (`(days with stops / total active days) × 100`). If the score is below 50, the explanation reads `"inconsistent stop placement"`.

Pass criteria: Score matches the expected ratio. Explanation text changes based on whether the score crosses the 50-point threshold.

TC-04 — STUDY PLAN: AGENT PATH WITH AUDIT RECORD

Tests: Agentic planning runs all 5 tools in order; LLM writes its own tasks; audit trail is written; `weeklyOutlook` is present.

1. Navigate to **Profile** (`/profile`) → **Readiness** tab
2. Review the readiness breakdown — confirm at least one component is below 60
3. Click **Generate Study Plan**
4. Wait for the plan to load (the agent calls 5 tools sequentially — this takes a few seconds)

What to look for: - A multi-session plan where each session has a specific `focus`, `task`, and `durationMinutes` - Task descriptions are written in the LLM's own words — not copied from the recommendation titles shown on screen - A **Weekly Outlook** section showing 2–4 week projections with milestones - A **Tool Trace** section (or expandable panel) showing 5 steps in order: `rank_readiness_gaps` → `get_drill_templates` → `build_seed_plan` → `estimate_weeks_to_target` → `score_plan`, each with `"status": "ok"` - An `auditId` field containing a UUID

Audit log: Open `data/responsible-ai/study-plan-tool-audit.ndjson` in any text editor. The last line should be a new JSON record containing `"status": "ok"`, `"toolCount": 5`, and `"source": "agent"`.

Pass criteria: Tool trace has exactly 5 entries in the correct order. Weekly outlook is present. Audit file has gained exactly one new line. `auditId` is a valid UUID.

TC-05 — STUDY PLAN: DETERMINISTIC FALLBACK

Tests: Responsible AI requirement — system functions without LLM.

1. Open `.env.local` and remove or blank the `OPENAI_API_KEY` value
2. Restart the app (`npm run dev`)
3. Navigate to **Profile** → **Readiness** tab and click **Generate Study Plan**

What to look for: - A study plan is still returned — no error page, no spinner that never resolves - The plan source is marked `"fallback"` (visible in the response or a UI label) - A `fallbackReason` field reads `"OPENAI_API_KEY not set or agent encountered an error"` - The tool trace is empty (`[]`) — the fallback runs pure Python, not the agent

Audit log: The new line in `data/responsible-ai/study-plan-tool-audit.ndjson` records `"source": "fallback"` and `"status": "ok"` — the fallback is logged, not silently swallowed.

Pass criteria: A usable plan is returned even with no API key. Student experience is uninterrupted. Fallback is audited identically to the agent path.

TC-06 — QUERY ROUTER: COMPLEXITY CLASSIFICATION

Tests: Router correctly distinguishes simple vs. complex queries.

1. Navigate to **Coaching** (`/coaching`)
2. Make sure the model selector is set to **Auto** (uses the DistilBERT router)
3. Send a simple query: `"What is a Sharpe ratio?"`
4. Open browser DevTools → **Network** tab → find the `/api/chat` request → inspect the response headers

What to look for (simple query): - Response header `x-router-label: simple` - Response header `x-router-confidence: 0.9x` (above 0.70)

1. Now send a complex query: `"Derive the Black-Scholes PDE from first principles using Ito's lemma and no-arbitrage"`

What to look for (complex query): - Response header `x-router-label: complex` - Confidence above 0.70

1. Send an ambiguous query: `"Derive hmm"`

What to look for (below-threshold): - Confidence below 0.70 → router defaults to `medium`, never routes to `complex`

Pass criteria: Simple queries route to the lighter model. Complex queries route to the stronger model. Ambiguous queries default to the safe middle tier rather than mis-routing.

TC-07 — QUIZ PROGRESS: PERSISTENCE AND RECOVERY

Tests: Problem question 4 — in-progress state survives page close.

1. Navigate to **Learn** (`/learn`) and open any quiz
2. Answer 3 questions — on any wrong answer, select a confidence level and a mistake type (`Conceptual`, `Careless`, or `Implementation`) before advancing
3. Close the browser tab entirely (do not click Submit or Finish)
4. Reopen the same quiz URL

What to look for: The quiz resumes at question 4. Questions 1–3 are shown as already answered with the original selections pre-filled. The progress indicator reflects 3/N completed, not 0/N.

Pass criteria: Quiz does not restart from question 1. Confidence and mistake-type annotations from the previous session are preserved.

TC-08 — RAG COACHING: CITATION AND CLAIM EVALUATION

Tests: Responsible AI requirement — coaching answers are verifiable, not just plausible.

1. Navigate to **Coaching** (`/coaching`)
2. Enable **PDF mode** (toggle or button in the coaching UI)
3. Upload any PDF (e.g., a lecture note or paper on volatility, options pricing, or another quant topic)
4. Wait for the indexing confirmation
5. Ask a question directly about the document's content, e.g.: `"What does this document say about volatility?"`

What to look for: - Inline citation markers in the answer text, formatted as `[c1.0.3]` (chunk references) - A citations panel listing each reference with page number, paragraph, and a snippet of the source text - A claim evaluation section where each factual claim in the answer is labeled `"supported"`, `"partial"`, or `"unsupported"` against the actual document chunks - A `confidence` score between 0 and 1

Pass criteria: Every inline citation marker maps to an entry in the citations list. No claim sourced from the document is marked `"unsupported"`. The system surfaces any hallucinated or ungrounded claim rather than hiding it.

TC-09 — SMART BETA ANALYSIS: FACTOR REGRESSION OUTPUT

Tests: ETF factor analysis runs and returns structured data with caching.

1. Navigate to the **Analysis** section (or the smart beta tool in the app)
2. Enter tickers: `USMV, MTUM`
3. Run the analysis — the first run fetches live data from yfinance and FRED (allow ~10 seconds)
4. Note the load time, then run the same tickers again immediately

What to look for (first run): - Factor loadings for each ticker: `alpha`, `MKT`, `SMB`, `HML` betas, and `r_squared` - `r_squared` values are between 0 and 1 - Rolling 36-month beta chart visible

What to look for (second run): - Response returns noticeably faster (under 1 second) — Parquet cache is serving the result

Pass criteria: Both tickers appear with valid factor loadings. `r_squared` is in range. The second call is significantly faster than the first.

TC-10 — AUDIT LOG INTEGRITY

Tests: Responsible AI requirement — append-only, every run recorded, all records valid JSON.

1. Open `data/responsible-ai/study-plan-tool-audit.ndjson` in a text editor — count the current number of lines
2. Navigate to **Profile** → **Readiness** tab and generate a study plan (TC-04)
3. Reopen the audit file

What to look for: - Exactly one new line has been appended — no lines removed or modified - The new line is valid JSON containing `"status": "ok"` and an `"auditId"` matching the one shown in the study plan UI - Every line in the file is valid JSON (you can scan visually or open with a JSON formatter)

Pass criteria: Line count increases by exactly 1 per plan generation. The audit file is strictly additive — it is never overwritten. Every record is parseable JSON.

TC-11 — READINESS COMPOSITE: CROSS-SIGNAL IMPACT

Tests: Problem question 3 — composite correctly identifies weakest component across signals.

Scenario: A learner with strong quiz scores but poor trading discipline.

1. Complete a few quizzes on the **Learn** page (aiming for mostly correct answers)
2. Complete an interview session on the **Profile** → **Interview** tab
3. Navigate to the **Dashboard** (`/`)

What to look for: - The readiness composite is broken into four visible components: Theory (30%), Implementation (25%), Execution (30%), Communication (15%) - If trading data is absent or discipline signals are low, the Execution component pulls the composite down significantly — even if quiz scores are high - The first recommendation on the dashboard targets the weakest component, not the highest-scoring one - Each

recommendation card shows a `source` tag (e.g., `/profile/resume`, `/profile/interview`, `/trade`) so the evidence link is traceable

Key point: A quiz-only system would return the Theory score alone (potentially 85+). QLOS returns a composite that reflects actual execution readiness. The difference between the two numbers is the platform's core value.

Pass criteria: Composite reflects all four components with correct weights. The weakest component's recommendation appears first. Source tags are visible on every recommendation card.

---

9. Results and Findings

FINDING 1: MULTI-SIGNAL SCORING REVEALS HIDDEN WEAKNESSES

The most significant finding from development is how dramatically behavioral signals shift the composite. A learner who scored 85% on quizzes but placed zero stop orders and made two revenge trades received: - Theory: 85 - Implementation: 65 - Execution: 18 - Composite: 55

The quiz-only signal: 85. The gap (85 vs 55) represents the difference between a student who knows the material and a student who is ready to use it under pressure. The composite surfaces this. Quiz score alone cannot.

FINDING 2: THE FALLBACK PLANNER IS CORRECT 100% OF THE TIME BECAUSE IT RUNS THE SAME LOGIC

The deterministic fallback implements the same gap-ranking algorithm as the agent's `rank_readiness_gaps` tool. In local testing, the fallback and agent paths produce structurally identical session plans. The agent adds richer rationale prose, LLM-authored task descriptions, a `weeklyOutlook` projection, and timeline estimates from `estimate_weeks_to_target`; the core structure is equivalent. This is the intended design: the agent adds quality, the fallback ensures availability. Neither can produce a misleading plan because both run the same ranking logic.

FINDING 3: THE DISTILBERT ROUTER IS ACCURATE ON CLEAR-CUT CASES AND CORRECTLY CONSERVATIVE ON AMBIGUOUS ONES

Simple definitional queries ("What is beta?") and complex multi-step derivations ("Prove the Fundamental Theorem of Asset Pricing") are classified with >85% confidence. The 0.70 confidence threshold is correctly calibrated: queries that fall below it (genuinely ambiguous phrasing) default to "medium" (gpt-4o-mini), which is the safe choice — it never mis-routes a complex query to a weak model.

FINDING 4: CITATION-GROUNDED COACHING CHANGES STUDENT BEHAVIOR

During testing, when answers included citation IDs linked to specific page/paragraph locations, students could cross-check claims directly. The LLM-as-a-Judge evaluation flagged one case where the model partially hallucinated a claim not present in the uploaded document — support was marked `"partial"` with a note. Without the evaluation, the hallucination would have been invisible.

FINDING 5: MISTAKE TAXONOMY REQUIRES ACTIVE STUDENT PARTICIPATION

The most important design tension: the mistake taxonomy is only as useful as the annotations students provide. In early testing, students skipped the annotation. The UI now defaults confidence to `"Med"` and prompts for mistake type immediately after revealing a wrong answer (before advancing to the next question). Completion rate of annotations improved significantly.

---

10. Observations and Honest Limitations

WHAT WORKS WELL

- The multi-signal readiness model produces clearly different (and more actionable) output than quiz-only scoring. This is the core value proposition and it delivers.
- The audit trail is genuinely useful for reviewing AI behavior. Inspecting the NDJSON log shows exactly what the agent decided and why.
- The fallback architecture means the app never breaks from an AI failure. Every failure mode has been tested and produces a usable result.
- Behavioral analytics from order history is a novel feedback mechanism — trading platforms don't usually loop execution behavior back into learning recommendations.

HONEST LIMITATIONS

No automated test suite. All test cases in this document are manual UI walkthroughs. There are no Jest tests, no pytest tests, no CI/CD pipeline. This is the largest gap for production readiness.

Readiness weights are designed, not calibrated. The 0.30 / 0.25 / 0.30 / 0.15 weights are based on domain judgment. There is no ground-truth readiness dataset to validate them against. A future version should collect outcomes data and regress weights from actual hire/pass rates.

Subprocess cold start. Spawning Python on each request adds 200–500ms latency. This is acceptable for planning (user expects a wait) but noticeable for the chat router on the first message. A persistent FastAPI sidecar would eliminate this in production.

LLM interview quality is model-dependent. GPT-4o produces significantly better firm-persona adherence than gpt-4o-mini. The default is gpt-4o-mini to manage cost; gpt-4o is recommended for actual interview prep use.

Auth is demo-grade. Passwords in plaintext SQLite, sessions in localStorage. Not for deployment.

DistilBERT router trained on ~150 bootstrapped queries. Performance on edge-case quant finance phrasing may degrade. A larger, human-labeled dataset would improve robustness.

11. API Reference

AUTH

ENDPOINT	METHOD	DESCRIPTION
`/api/auth/register`	POST	Register new user
`/api/auth/login`	POST	Authenticate
`/api/auth/user`	GET	Current session user

LEARNING

ENDPOINT	METHOD	DESCRIPTION
`/api/learn/quiz-progress`	GET/POST	Read/write in-progress quiz state

PROFILE

ENDPOINT	METHOD	DESCRIPTION
`/api/profile/readiness/study-plan`	POST	Generate agentic study plan (with audit)
`/api/profile/resume`	GET/POST	Upload/retrieve resume
`/api/profile/resume/analysis`	POST	Store extracted analysis
`/api/profile/resume/file`	GET/POST	Resume file operations

INTERVIEW

ENDPOINT	METHOD	DESCRIPTION
`/api/interview`	POST	Single LLM interview turn
`/api/interview/results`	GET/POST	Session history

COACHING

ENDPOINT	METHOD	DESCRIPTION
`/api/chat`	POST	Streaming chat (supports `model: "auto"`)
`/api/coaching/rag`	POST	Document index + cited query
`/api/coaching/conversations`	GET/POST/DELETE	Conversation history

TRADING AND ANALYTICS

ENDPOINT	METHOD	DESCRIPTION
`/api/alpaca/account`	GET	Account balance and buying power
`/api/alpaca/orders`	GET/POST	List / place orders
`/api/alpaca/orders/[orderId]`	DELETE	Cancel order
`/api/alpaca/positions`	GET	Open positions
`/api/alpaca/bars/[symbol]`	GET	Price bars for charting
`/api/alpaca/sessions`	GET	Trading session history
`/api/alpaca/behavioral`	GET	Compute behavioral signals
`/api/alpaca/performance`	GET	P&L summary
`/api/analysis/smart-beta`	GET	Factor regression (with caching)

12. Local Development

Prerequisites: Node.js 20+, Python 3.10+, npm, pip

```
git clone https://github.com/alexksh2/Deep_Learning_Week_2026.git
cd Deep_Learning_Week_2026

# Install JS dependencies
npm install

# Python virtual environment
python3 -m venv .venv
source .venv/bin/activate

# Install Python dependencies (each pipeline is independent)
pip install -r router/requirements.txt
pip install -r ml-development/etf-analysis/requirements.txt
pip install -r ml-development/rag-pipeline/requirements.txt
pip install -r ml-development/interview-pipeline/requirements.txt

# Environment configuration
cp .env.local.example .env.local
# Required: OPENAI_API_KEY
# Required: ALPACA_API_KEY, ALPACA_SECRET_KEY (for trading dashboard, behavioral analytics, and
portfolio features)
#   Sign up at https://alpaca.markets → Paper Trading → API Keys

# Start application
npm run dev
# App runs at http://localhost:3000
```

Demo credentials: `alexkhoo@gmail.com` / `demo1234`

Useful diagnostic commands:

```
# View last 20 audit log entries
tail -n 20 data/responsible-ai/study-plan-tool-audit.ndjson

# Test query router directly
cd router && python router.py "Explain the Kelly criterion"

# Test behavioral signal computation (returns fallback if Alpaca not configured)
curl http://localhost:3000/api/alpaca/behavioral | python3 -m json.tool

# Check SQLite schema and data
sqlite3 data/auth.sqlite ".tables"
sqlite3 data/auth.sqlite "SELECT COUNT(*) FROM quiz_progress;"
```

Train the query router (optional — pre-trained checkpoint included if present):

```
cd router
python bootstrap_labels.py   # auto-label seed data with local Ollama
python prepare_data.py       # tokenize and split
python train.py              # fine-tune DistilBERT (~5 min CPU)
```

13. Repository Map

```
app/
├── api/                    23 server endpoints
│   ├── alpaca/             Alpaca proxy + behavioral analytics
│   ├── analysis/           Smart beta factor analysis
│   ├── auth/               Authentication
│   ├── chat/               Streaming chat + router
│   ├── coaching/           RAG pipeline endpoint
│   ├── interview/          LLM interview turns + history
│   ├── learn/              Quiz progress persistence
│   └── profile/            Resume, readiness, study-plan (with audit)
├── coaching/page.tsx       Chat UI with streaming + RAG mode
├── learn/                  Quiz and course pages
├── profile/page.tsx        7-tab profile workspace
├── trade/                  Simulator and analysis pages
└── page.tsx                Dashboard (readiness + recommendations)

components/
├── profile/                Resume, Aspirations, Interview, Readiness, Skills tabs
└── ui/                     Radix UI primitives

ml-development/
├── readiness-agent/        Agentic planner + deterministic fallback
├── interview-pipeline/     Firm-specific LLM interviewer
├── resume-analysis/        Skill extraction (70+ taxonomy)
├── rag-pipeline/           FAISS RAG + citation enrichment + claim evaluation
└── etf-analysis/           Factor regression + Parquet caching

router/
├── bootstrap_labels.py     Auto-label seed data with Ollama
├── prepare_data.py         Tokenize and split
├── train.py                Fine-tune DistilBERT
├── router.py               Inference (classify + route)
└── router_cli.py           Subprocess wrapper

lib/
├── types.ts                All TypeScript domain types
├── auth-db.ts              SQLite wrapper
├── quiz-question-bank.ts   100+ questions across 6 categories
└── quiz-progress.ts        Progress persistence helpers

data/
├── auth.sqlite             Users, quiz progress, interview results, conversations
├── resumes/                Uploaded resume files
└── responsible-ai/
    └── study-plan-tool-audit.ndjson    Append-only audit log

docs/
├── responsible-ai-framework.md
└── responsible-ai-study-plan-tool-logging.md
```

Every claim in this document maps to a specific file path, endpoint, or log entry. Responsible AI gaps are documented explicitly — honest enumeration of limitations is a requirement, not a weakness.